

Настройка IDE Android Studio

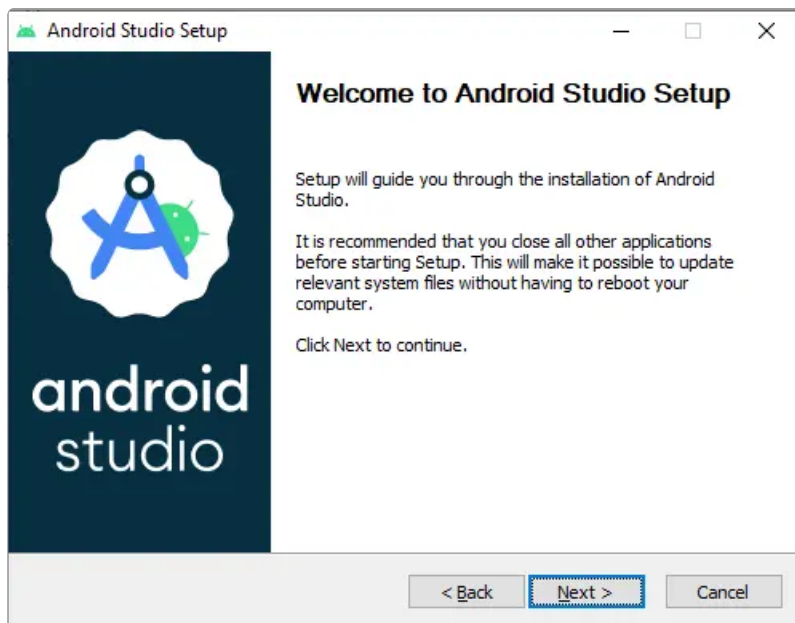
Установка

- 🔔 Необходимо устанавливать версию Android Studio, указанную в [группе мобильной разработки](#). Данная версия гарантировано подходит для сборки и отладки наших приложений.
- Остальные версии пока еще не проверялись и могут содержать баги, препятствующие нормальной работе разработчика.
- Для получения списка версий необходимо перейти на [сайт Android Studio](#), пролистать Соглашение до конца страницы и нажать "I agree the terms".

Windows

Предполагается, что предварительно выполнена базовая [настройка рабочего места в Windows](#).

1. Скачайте установочный файл IDE **Android Studio** [с официального сайта](#).
2. Запустите установочный файл и следуйте инструкциям на экране.



3. Запустите IDE из меню "Пуск".

Linux

Предполагается, что предварительно выполнена базовая настройка рабочего места в [RedHat](#).

1. Скачайте архив с IDE **Android Studio** [с официального сайта](#).
2. Распакуйте архив в домашней директории, например,

~/AndroidStudio

3. Для запуска IDE откройте файл **studio.sh** из каталога **android-studio** → **bin**.

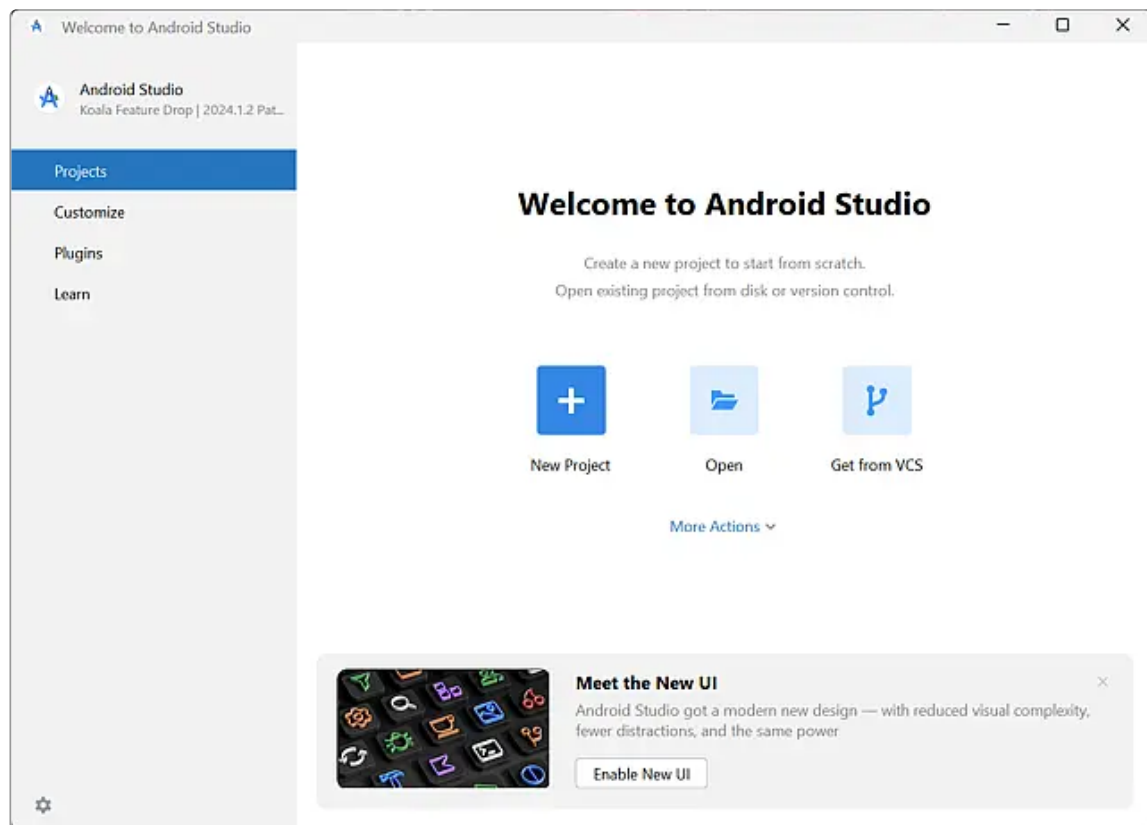
macOS

Предполагается, что предварительно выполнена базовая настройка рабочего места в [macOS](#).

1. Скачайте dmg-образ IDE **Android Studio** [с официального сайта](#).
2. Для запуска IDE откройте dmg-образ двойным кликом.

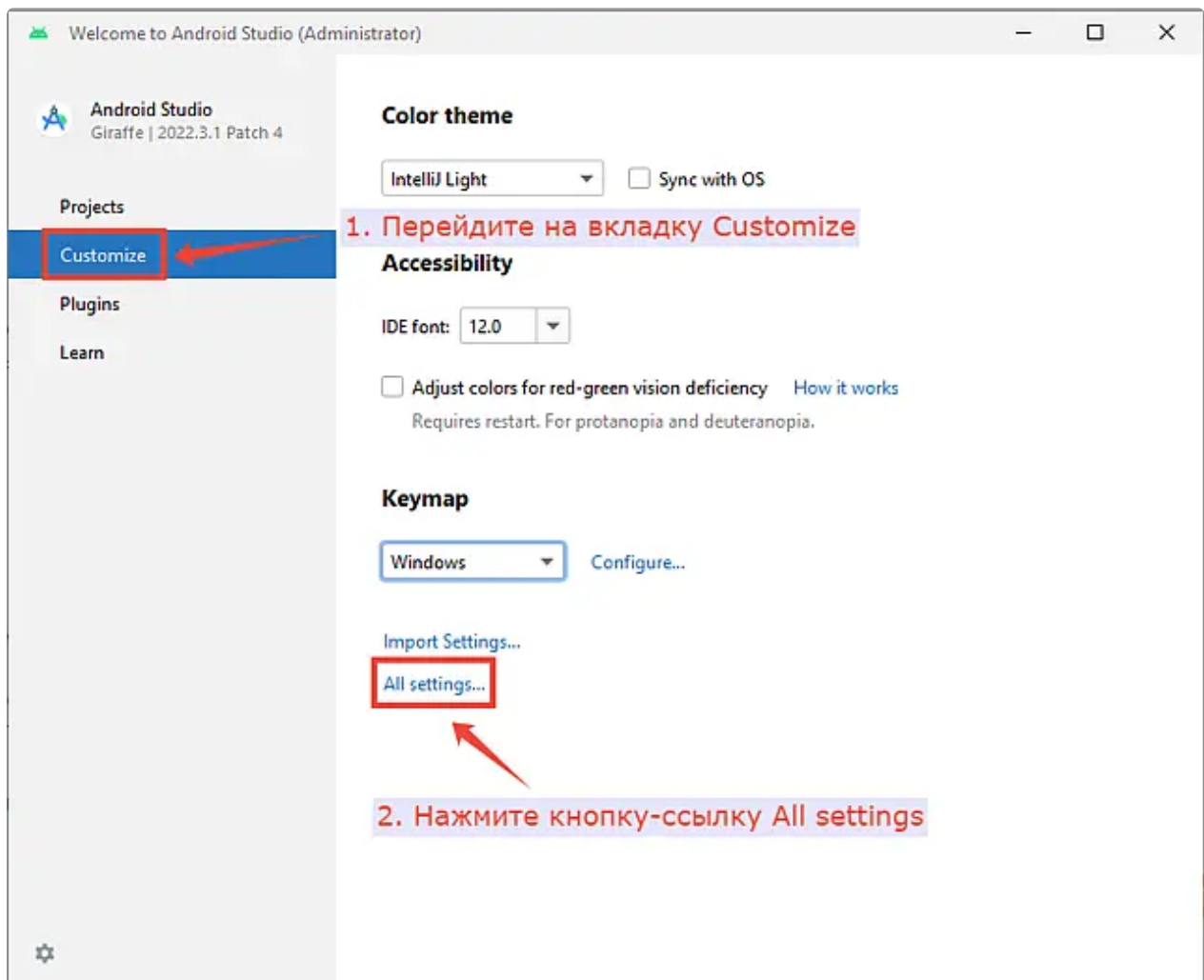
Первый запуск IDE

1. Откройте IDE **Android Studio**.
2. При первом запуске IDE будет открыто окно **Welcome to Android Studio**.

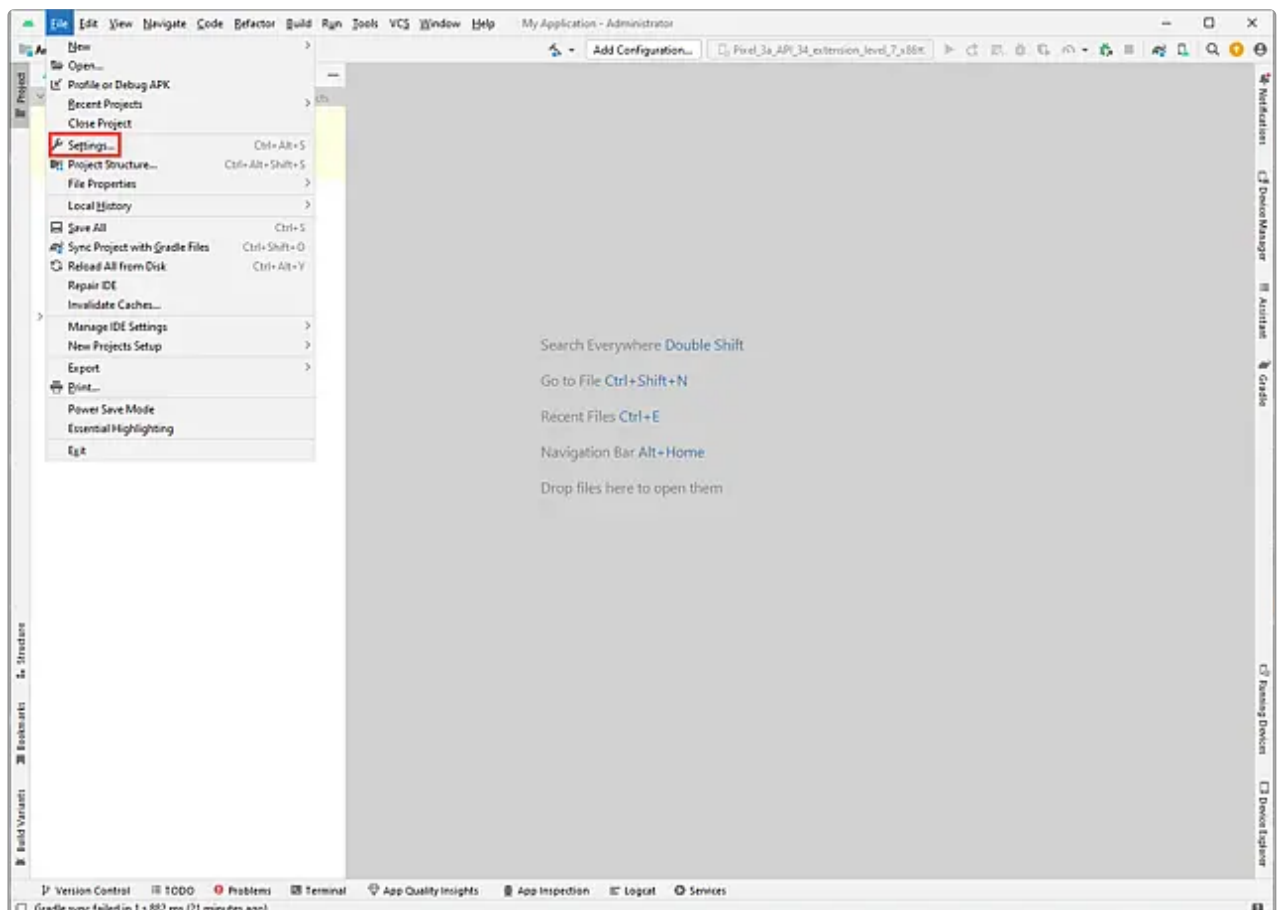


Установка SDK и NDK

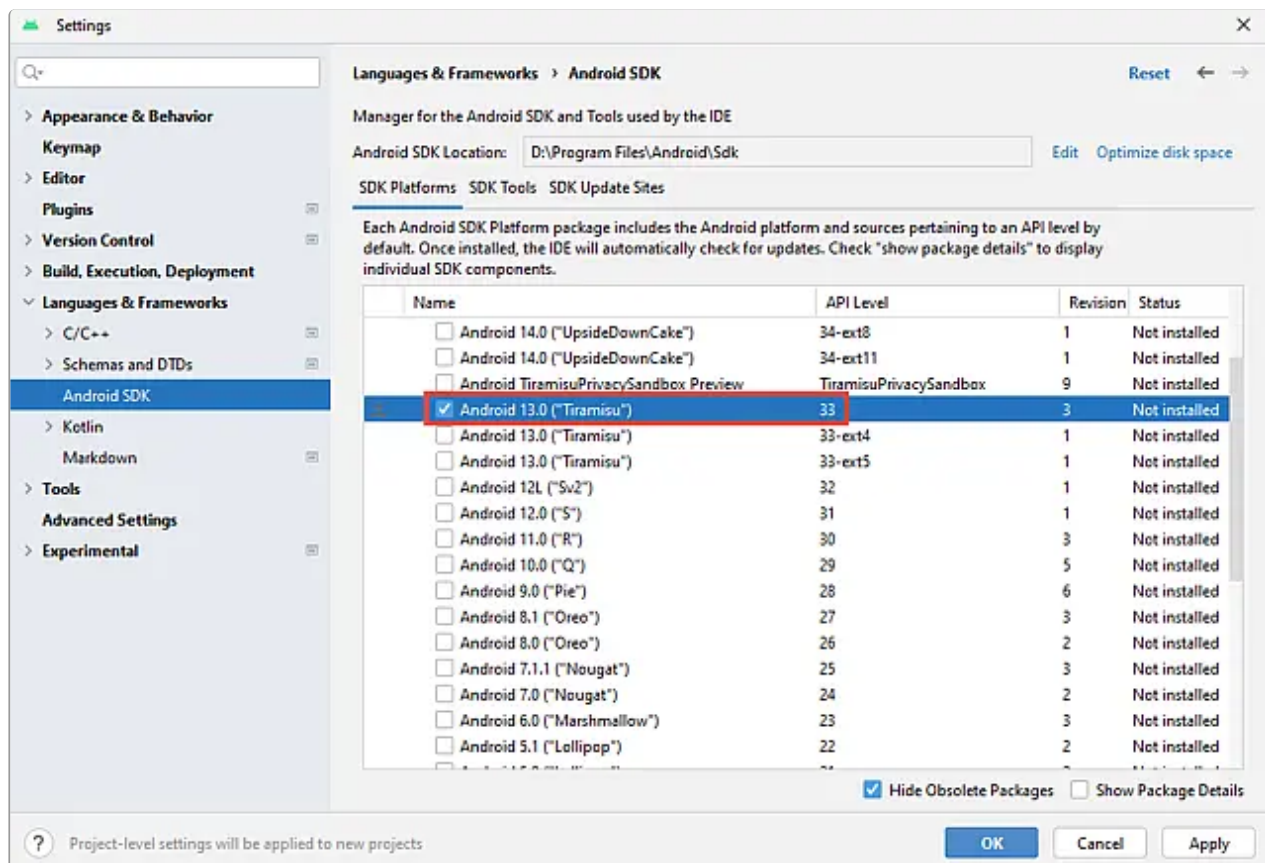
1. В IDE **Android Studio** в окне **Welcome to Android Studio** перейдите на вкладку **Customize** и нажмите кнопку-ссылку "All settings...".



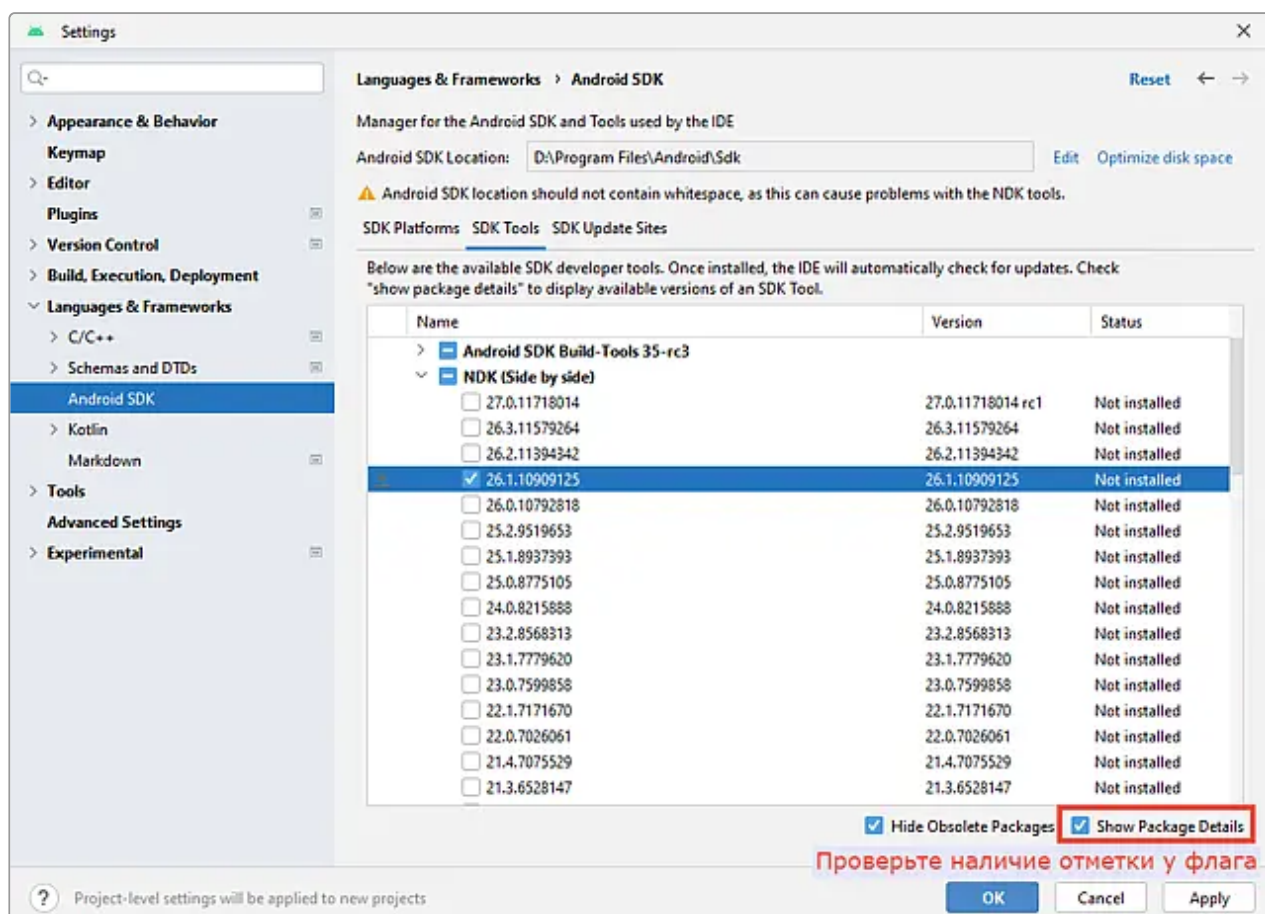
Если в IDE был открыт проект, тогда откройте меню **File → Settings**.



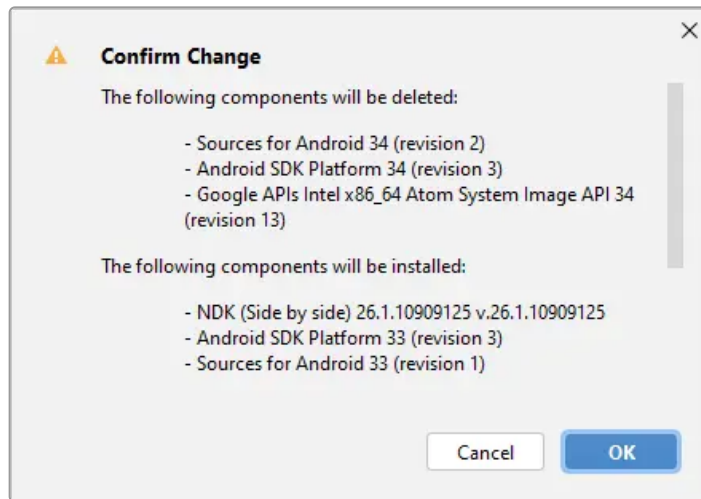
2. В аккордеоне диалогового окна **Settings** перейдите в пункт **Languages & Frameworks → Android SDK**. Также можно воспользоваться строкой поиска. В рабочей области на вкладке **SDK Platforms** в списке версий Android выберите нужную.



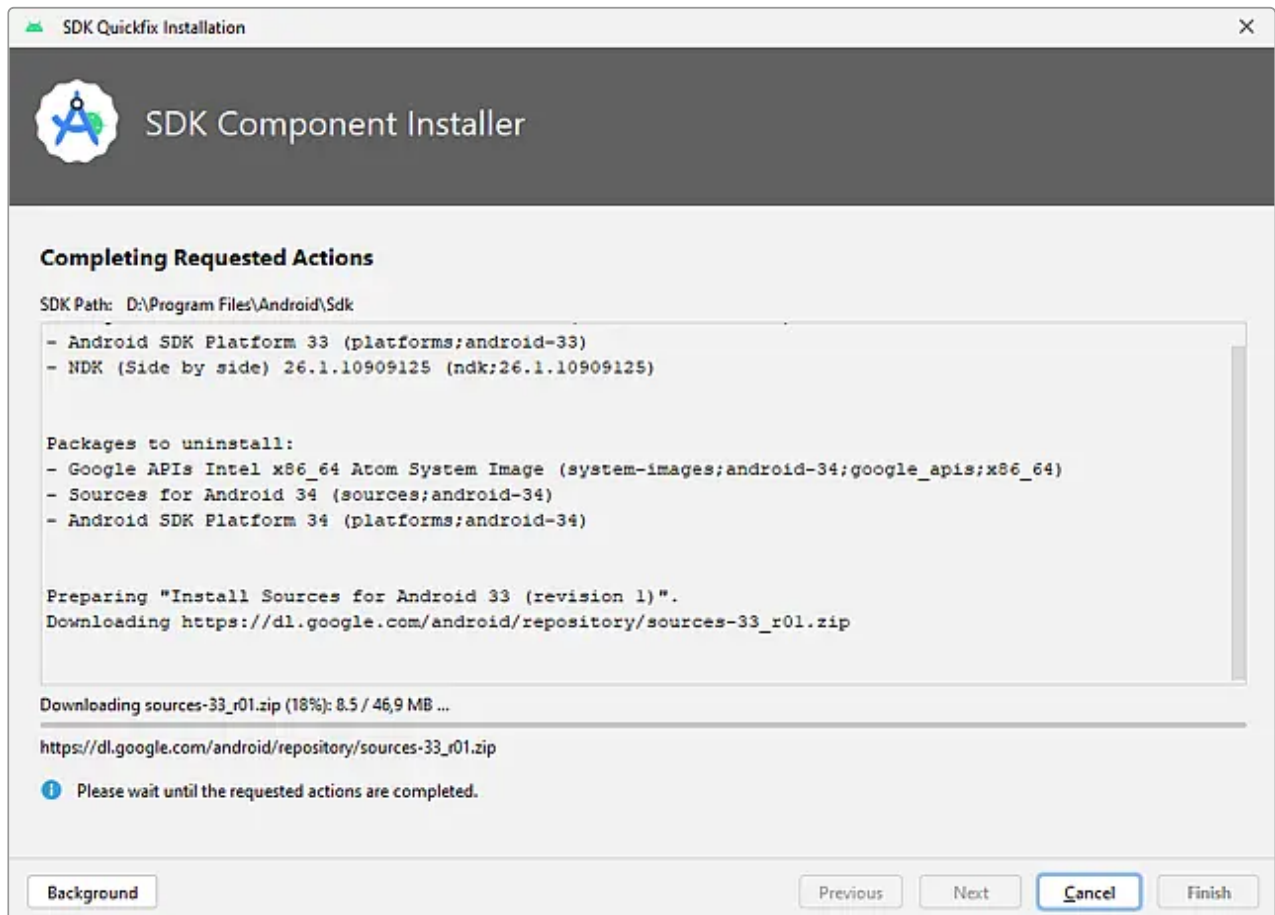
3. Перейдите на вкладку **SDK Tools** и установите версию NDK, указанную в [группе мобильной разработки](#). Проверьте наличие отметки у флага **Show Package Details**.



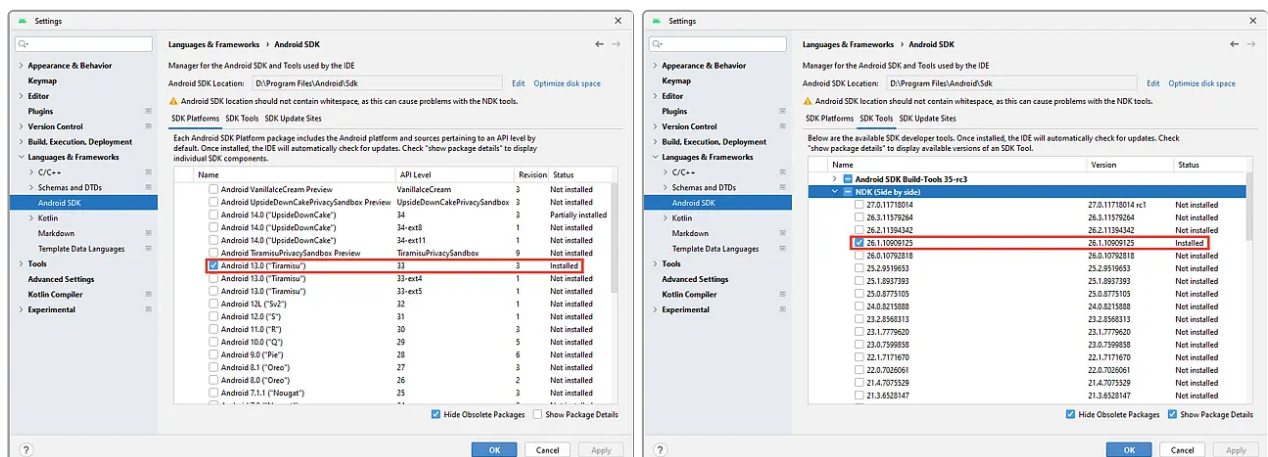
4. Далее нажмите кнопку "Apply". После этого появится окно **Confirm Change**. Подтвердите установку перечисленных компонентов и нажмите кнопку "OK".



5. После этого начнется шаг **SDK Component Installer** с установкой новых компонентов.



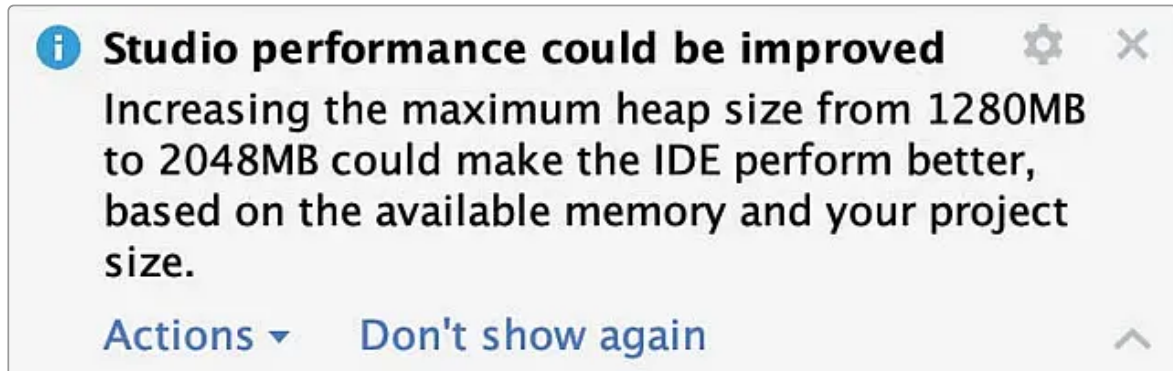
6. После установки компонентов в **Android Studio** перейдите в меню **File → Settings → Languages & Frameworks → Android SDK** и проверьте, что на вкладках **SDK Platforms** и **SDK Tools** для установленных компонентов отображается статус **Installed**.



Настройка памяти

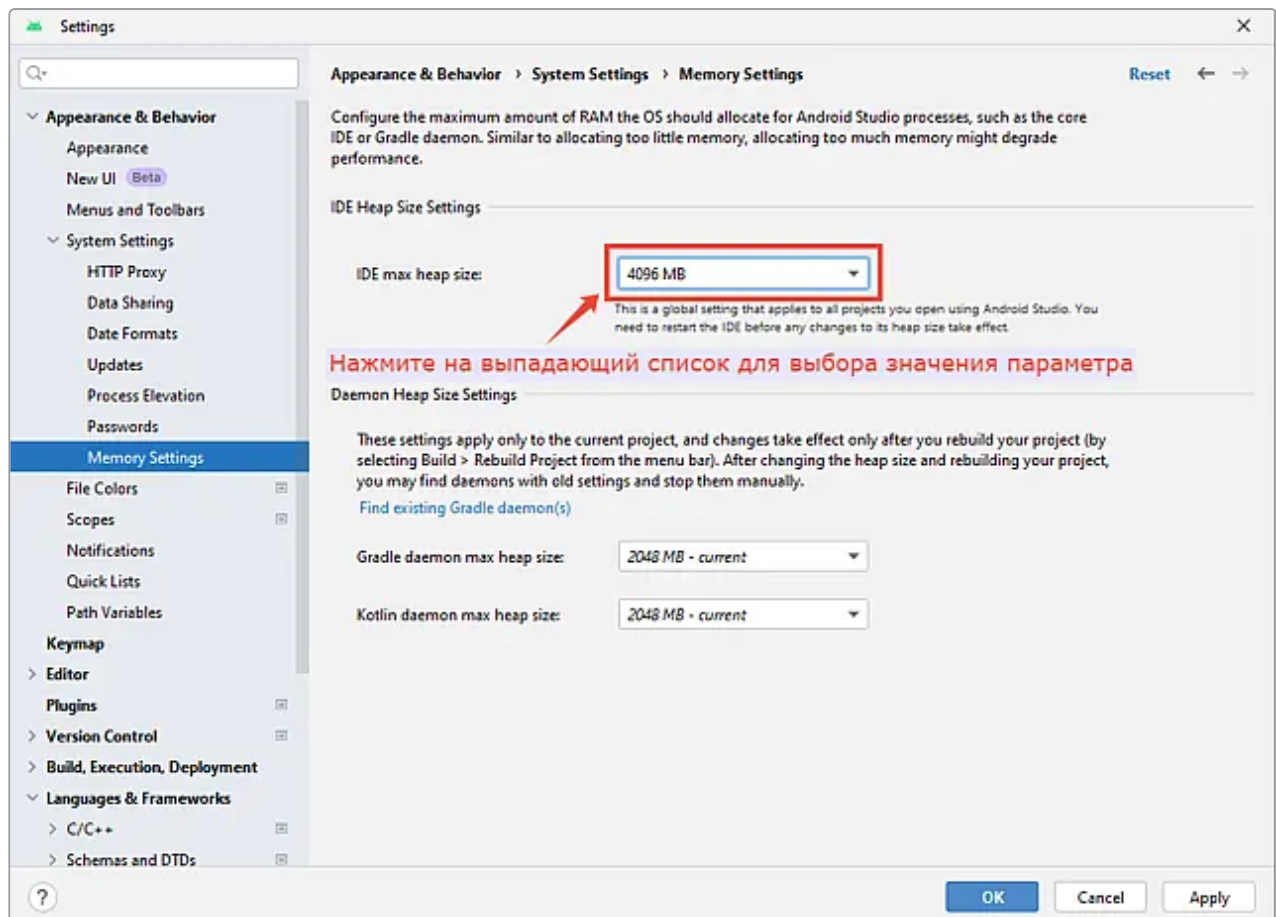
По умолчанию **Android Studio** имеет максимальный размер кучи (heap size) 1280 МБ. Если вы работаете над большим проектом или в вашей системе много оперативной памяти, вы можете повысить производительность, увеличив максимальный размер кучи для процессов **Android Studio**, таких как ядро IDE, демоны **Gradle** и **Kotlin**.

Android Studio автоматически проверяет возможную оптимизацию размера кучи и уведомляет вас, если обнаруживает, что производительность может быть улучшена.



Если вы используете 64-разрядную систему с объемом оперативной памяти не менее 5 ГБ, вы также можете настроить размеры кучи для своего проекта вручную. Для этого выполните следующие действия:

1. Нажмите **File** → **Settings** в строке меню (или **Android Studio** → **Settings** в macOS).
2. Перейдите на вкладку **Appearance & Behavior** → **System Settings** → **Memory Settings**. Отрегулируйте размеры кучи, чтобы они соответствовали желаемому количеству и нажмите кнопку "Apply":



Если вы изменили размер кучи для IDE, необходимо перезапустить **Android Studio**, прежде чем будут применены новые параметры памяти.

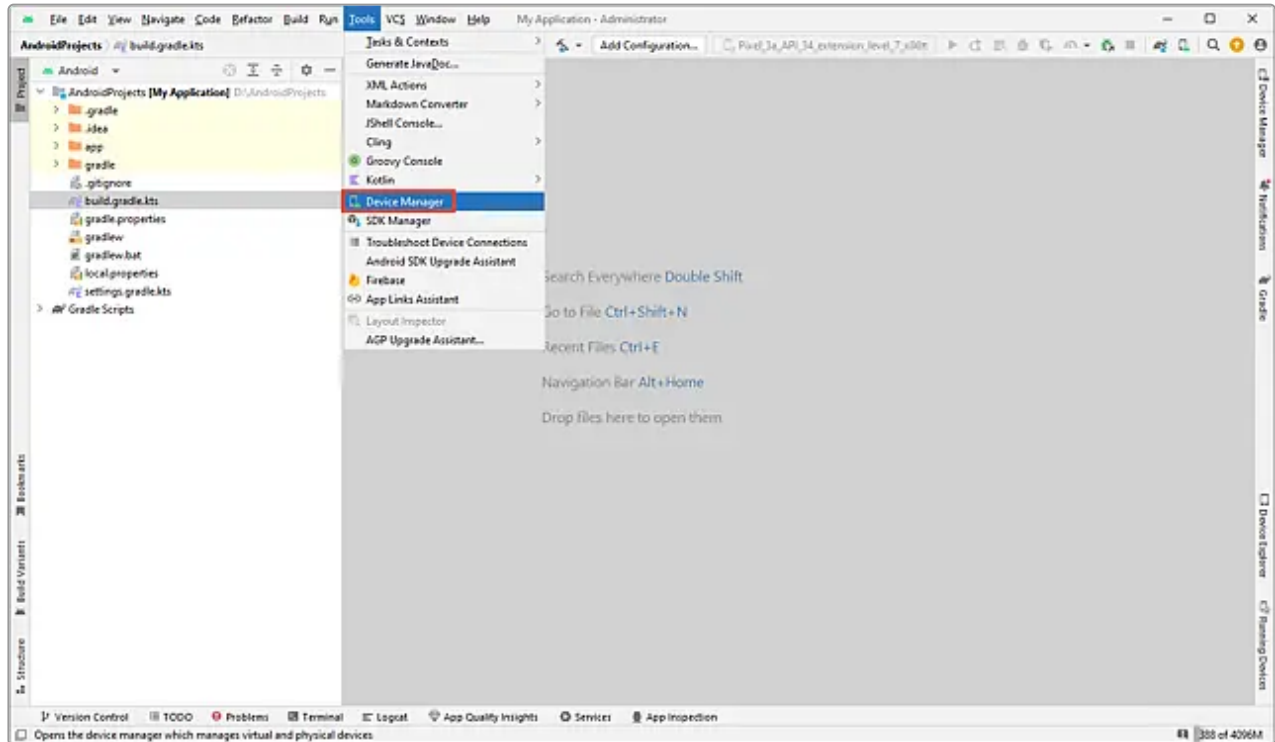
Настройка эмулятора



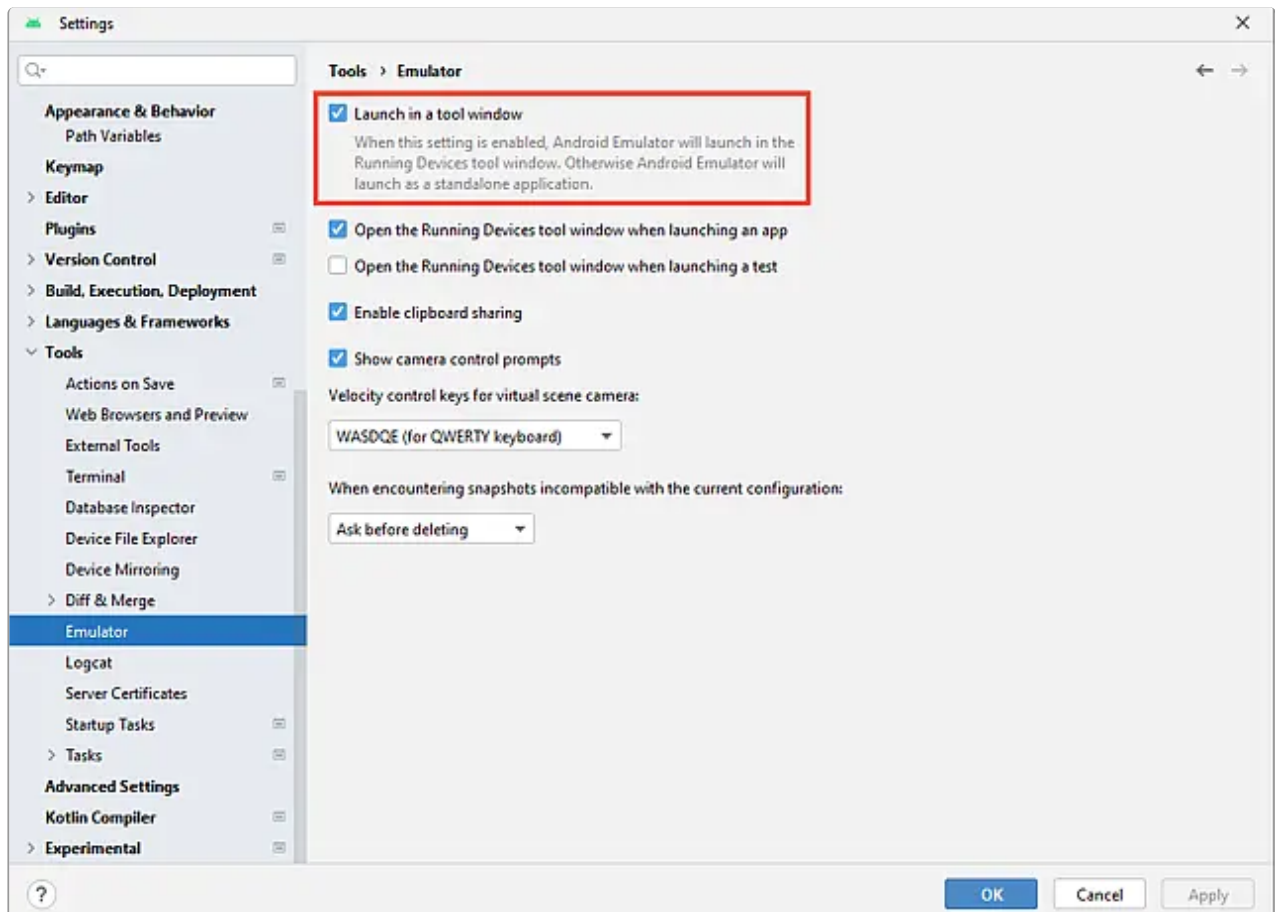
Важно!

На всех компьютерах, подключенных к офисной сети, проводится SSL-инспекция для проверки трафика (к каким ресурсам обращаются пользователи). При такой инспекции проводится подмена SSL-сертификата. Эмулятор **Android Studio** при запуске обращается к внешним сайтам и обнаруживает подмену. В результате эмулятор не может работать. Чтобы исключить ваш компьютер из SSL-инспекции, оформите поручение на отдел "[Эксплуатация Тензора](#)". В описании к поручению укажите причину исключения ("Для работы в Android Studio").

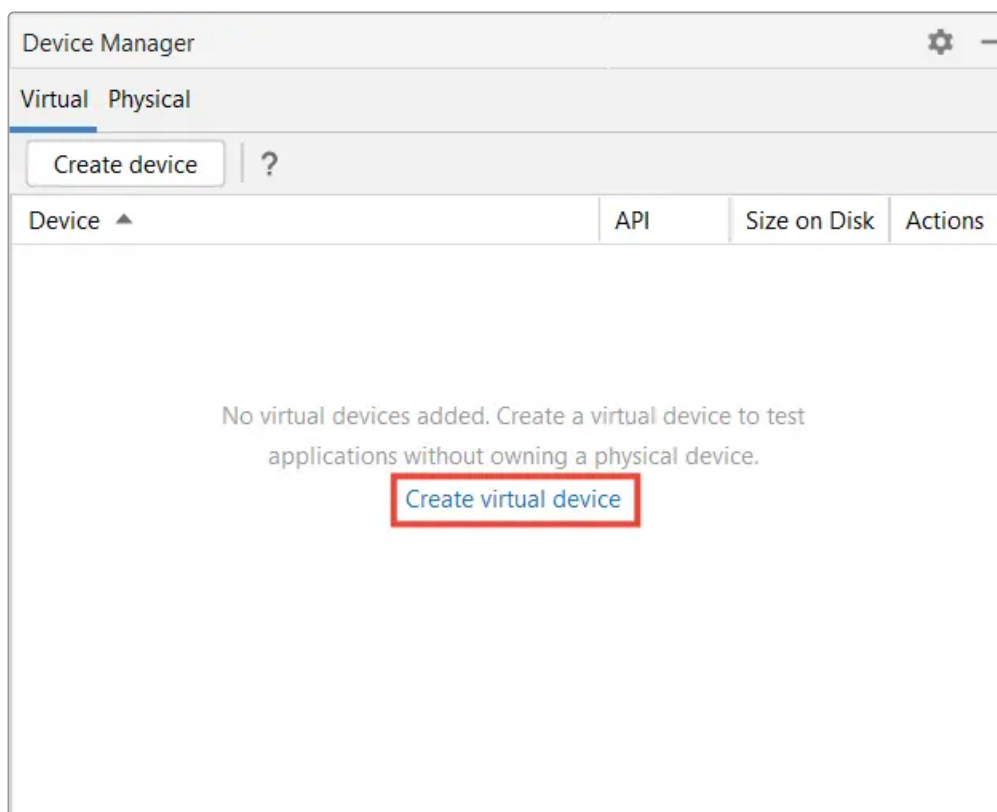
1. В IDE **Android Studio** откройте меню **Tools** → **Device Manager**.



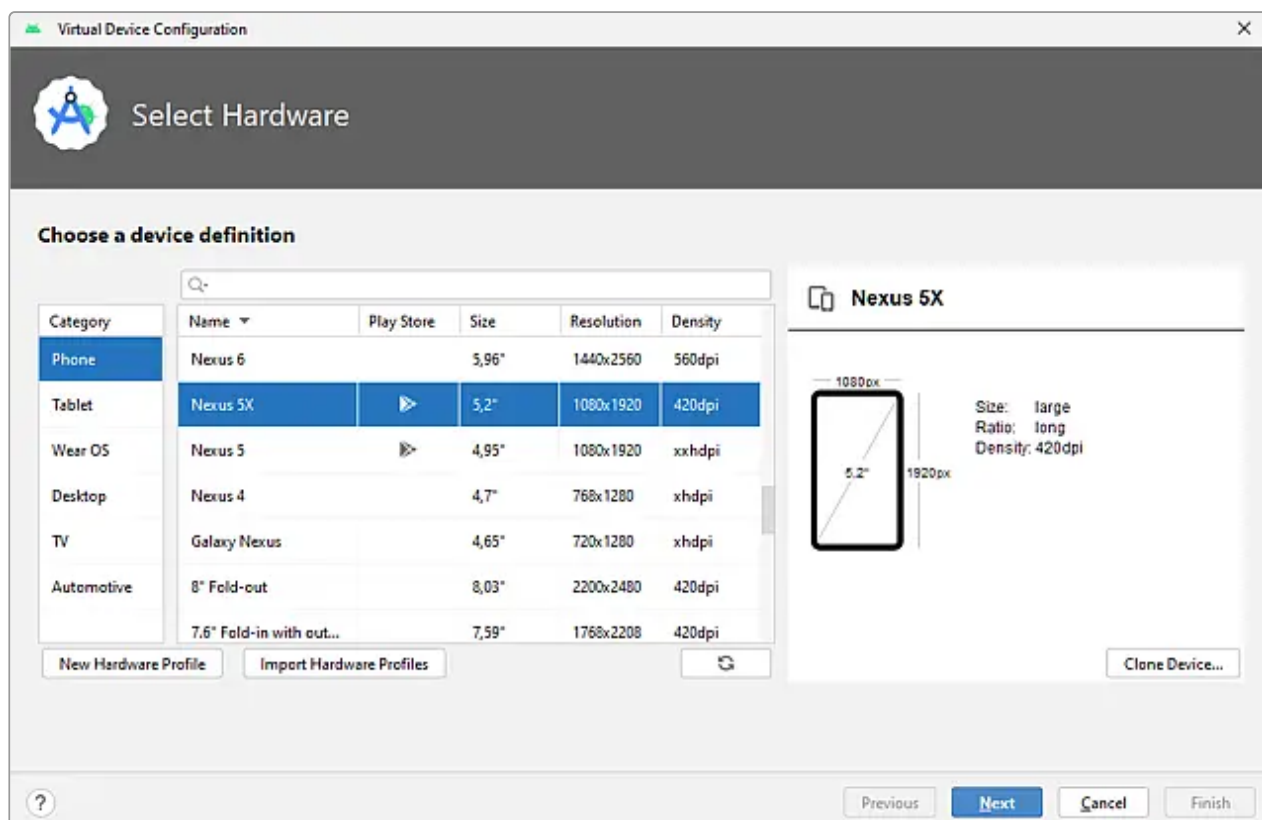
Если в последней версии IDE **Android Studio** в меню **Tools** нет **Device Manager**, отметьте в настройках флаг напротив опции **Launch in the Running Devices tool window** (можно найти в меню **Settings** → **Tools** → **Emulator**). Иначе эмулятор нужно запускать как standalone-приложение.



2. Откроется вкладка **Device Manager**. Изначально список устройств пуст. Нажмите кнопку-ссылку "**Add a new device...**", чтобы начать настройку добавляемого устройства.



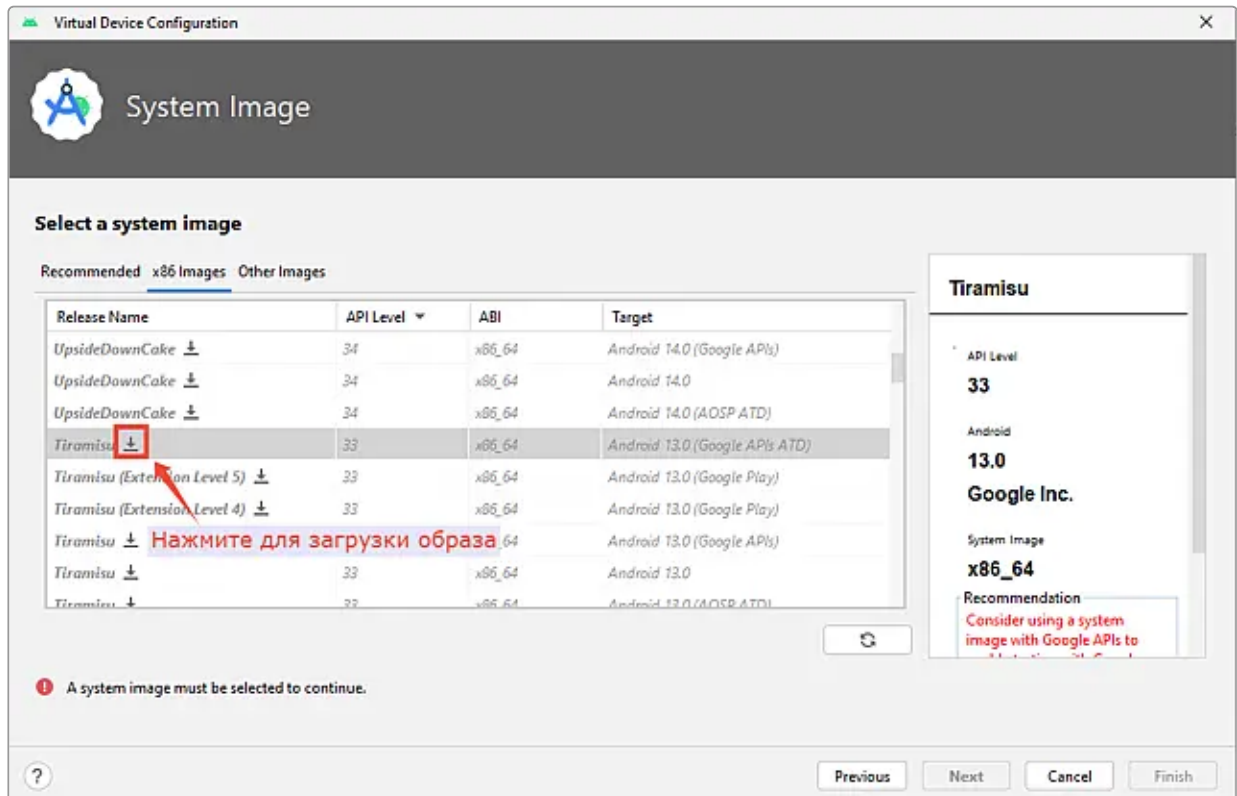
3. На этапе **Select Hardware** выберите категорию и тип устройства. Далее нажмите кнопку "Next".



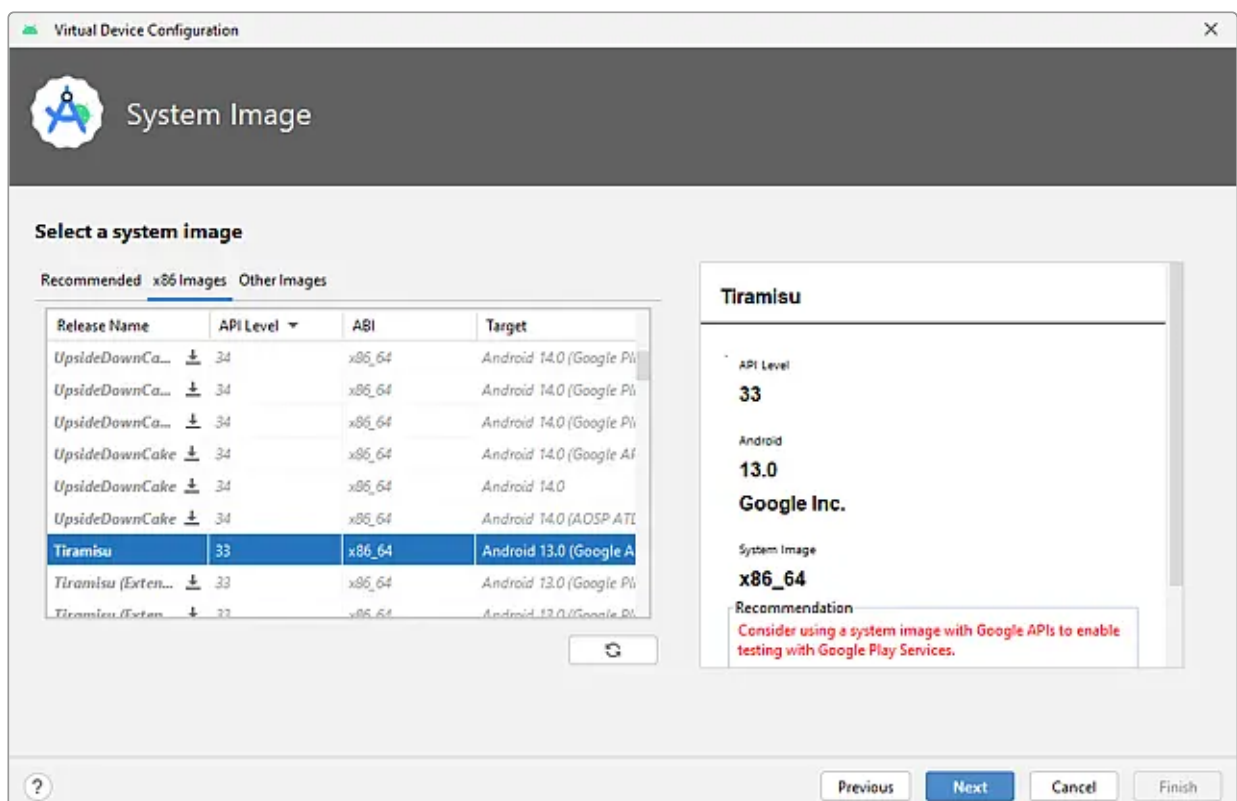
4. На следующем этапе **System Image** перейдите на вкладку **x86 Images** и выберите пункт **Tiramisu** (API Level - 33, ABI - x86_64, Target - Android Tiramisu).

- x86 может потребоваться если нужно тестировать legacy-приложения, есть проблемы совместимости с x86_64 или у вас старое оборудование. Мы переходим на вкладку **x86 Images**, чтобы отобрать эмуляторы под 64бит архитектуру. Это обеспечит лучшую производительность на современных компьютерах с Intel/AMD процессорами. Если у вас macOS, то выбираем ABI - ARM.
- Выбираем **Tiramisu** (API Level - 33). Выбор API зависит от минимально поддерживаемого приложением API и ваших задач.
- Выбор target влияет на набор функциональности эмулятора. Мы выбираем **Google APIs**.

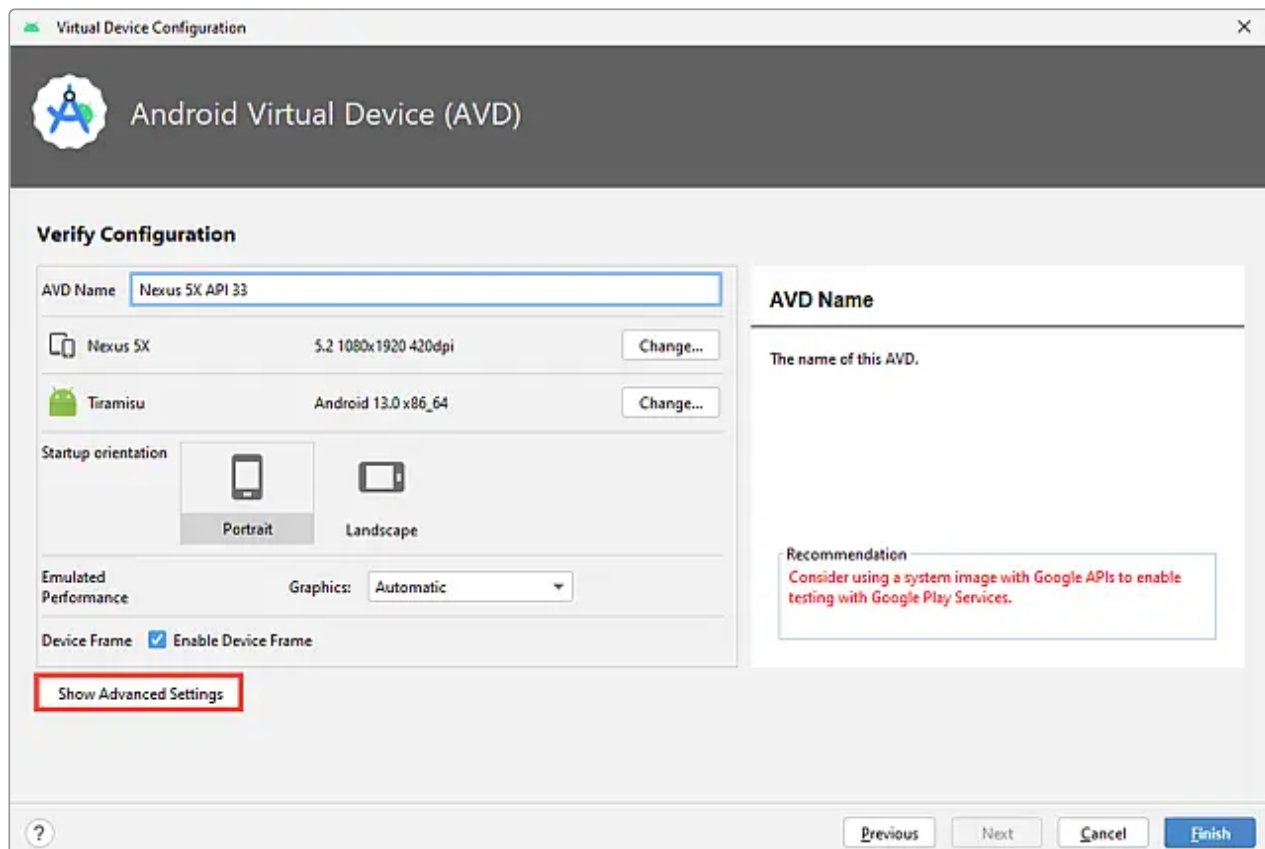
- **Google Play** - полноценная система, приближенная к реальному устройству, есть все сервисы google и приложения (в том числе маркет), сертифицирован google, требует больше ресурсов. Подойдет для финального тестирования приложения максимально приближенному к реальному устройству.
- **Google APIs** - есть google сервисы, но нет маркета и самих google приложений. Более легковесный, чем GooglePlay. Подойдет для разработки, когда вам достаточно доступа к api сервисов google, а не сами приложения. (Например google Maps).
- **Default Android System Image (AOSP)/ пустые скобки рядом с версией** - чистый android без доступа к google сервисам, минимальный набор базовых приложений. Подходит для разработки, в которой не требуются сервисы от google.
- **Google APIs или AOSP с подписью Automated Test Device (ATD)** - оптимизированные для тестирования, более урезанный функционал. Подходит, когда нужно быстро прогнать тесты.



Когда установка будет завершена, на этапе **System Image** для пункта **UpsideDownCake** пропадет значок загрузки. Далее нажмите кнопку "Next".



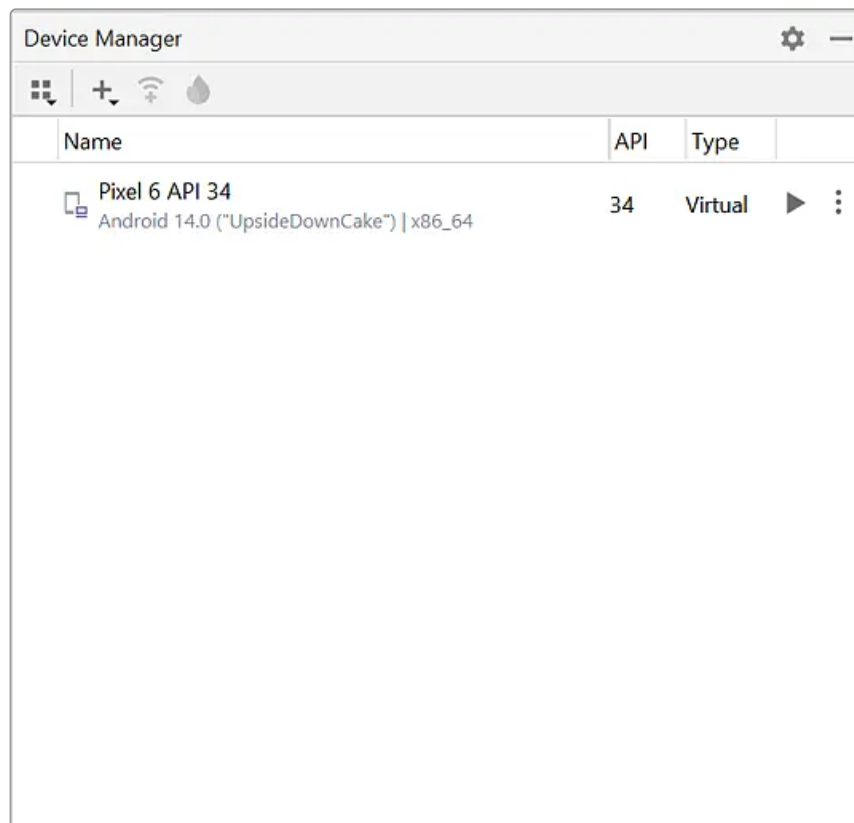
5. На этапе **Android Virtual Device (AVD)** нажмите кнопку "Show Advanced Settings", чтобы получить доступ к дополнительным настройкам.



Найдите следующие разделы и измените для них настройки (если для вашего оборудования это возможно):

- В разделе **Emulated Performance** убедитесь, что установлен флаг **Multi-core CPU**. Из выпадающего списка выберите максимальное доступное значение, например 2 или 4.
- В разделе **Memory and storage** измените следующие опции:
 - **RAM:** 2048 MB
 - **VM heap:** 512 MB
 - **Internal Storage:** 5000 MB

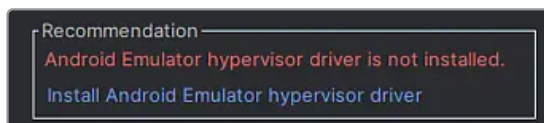
6. После выполнения этих настроек нажмите кнопку "Finish", в списке эмуляторов появится добавленное устройство. На этом добавление эмулятора устройства завершается. Аналогично выполняется добавление любого другого устройства.



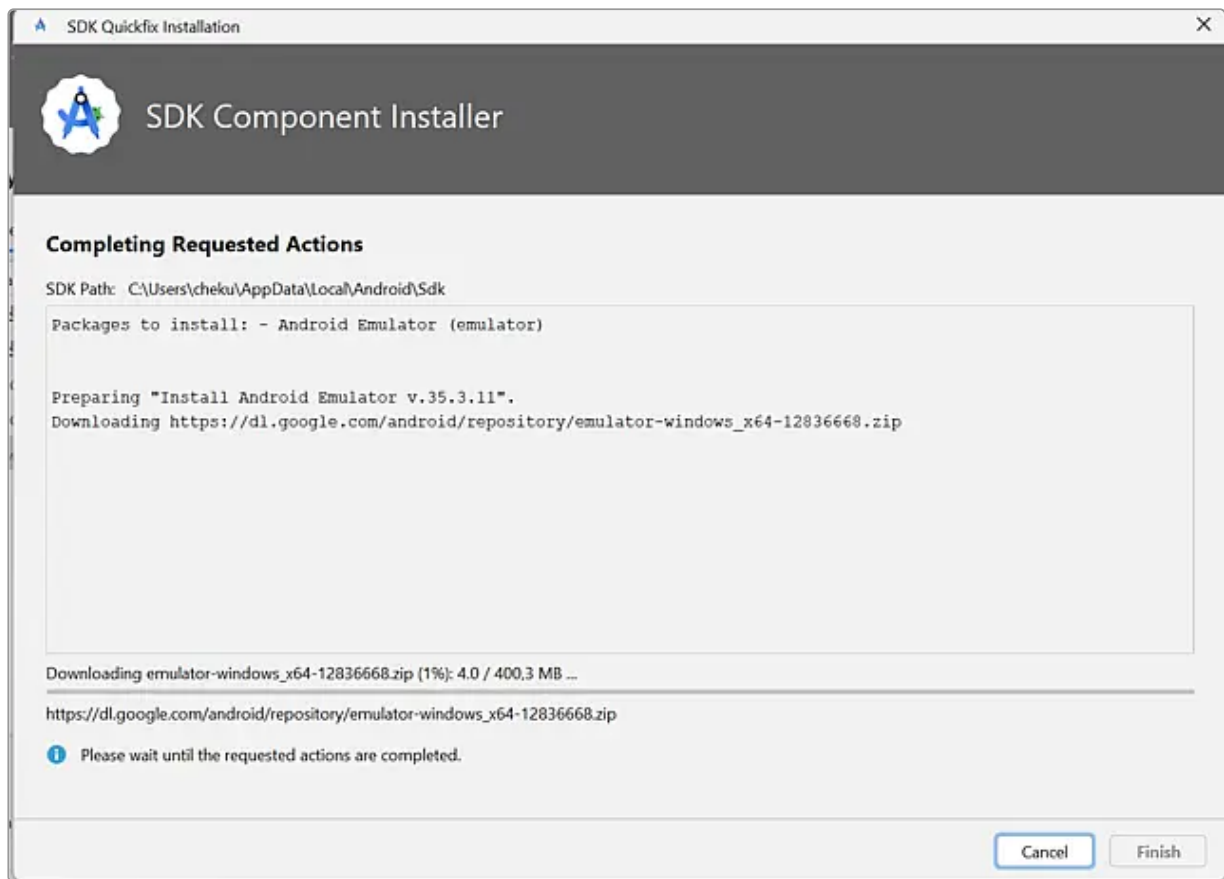
Ошибки и рекомендации в процессе создания эмулятора

Ошибки по загрузке недостающих утилит

Первый вид рекомендаций — это отсутствующие утилиты эмулятора, которые требуются для запуска:



Клик по кнопке-ссылке поможет вам быстро устранить проблему, скачать и установить все то, что необходимо для запуска эмулятора.



Ошибки выключенной виртуализации

Если на вашем рабочем/личном ПК на Linux или Windows ранее не запускалась виртуальная среда, то при настройке эмулятора вы можете встретить подобные предупреждения:

Recommendation HAXM is not installed. Install HAXM	Recommendation VT-x is disabled in BIOS. Troubleshoot
--	---

Ошибки такого рода связаны с выключенной настройкой аппаратной виртуализации в BIOS, для их решения воспользуйтесь [инструкцией для включения виртуализации](#). На macOS виртуализация включена по умолчанию.

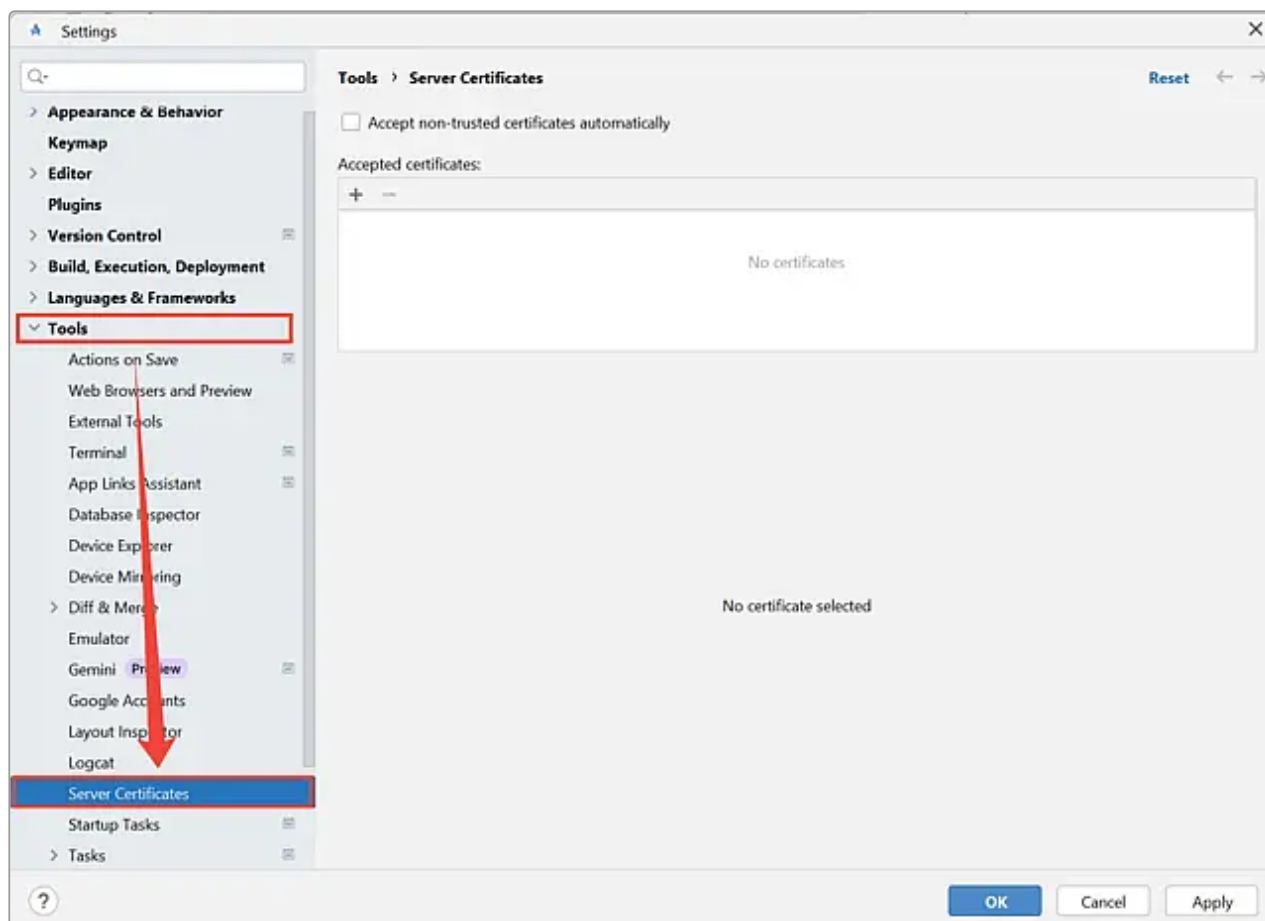
Настройка сертификатов

 Данные настройки требуются только при работе с Android Studio из внутренней сети филиалов.

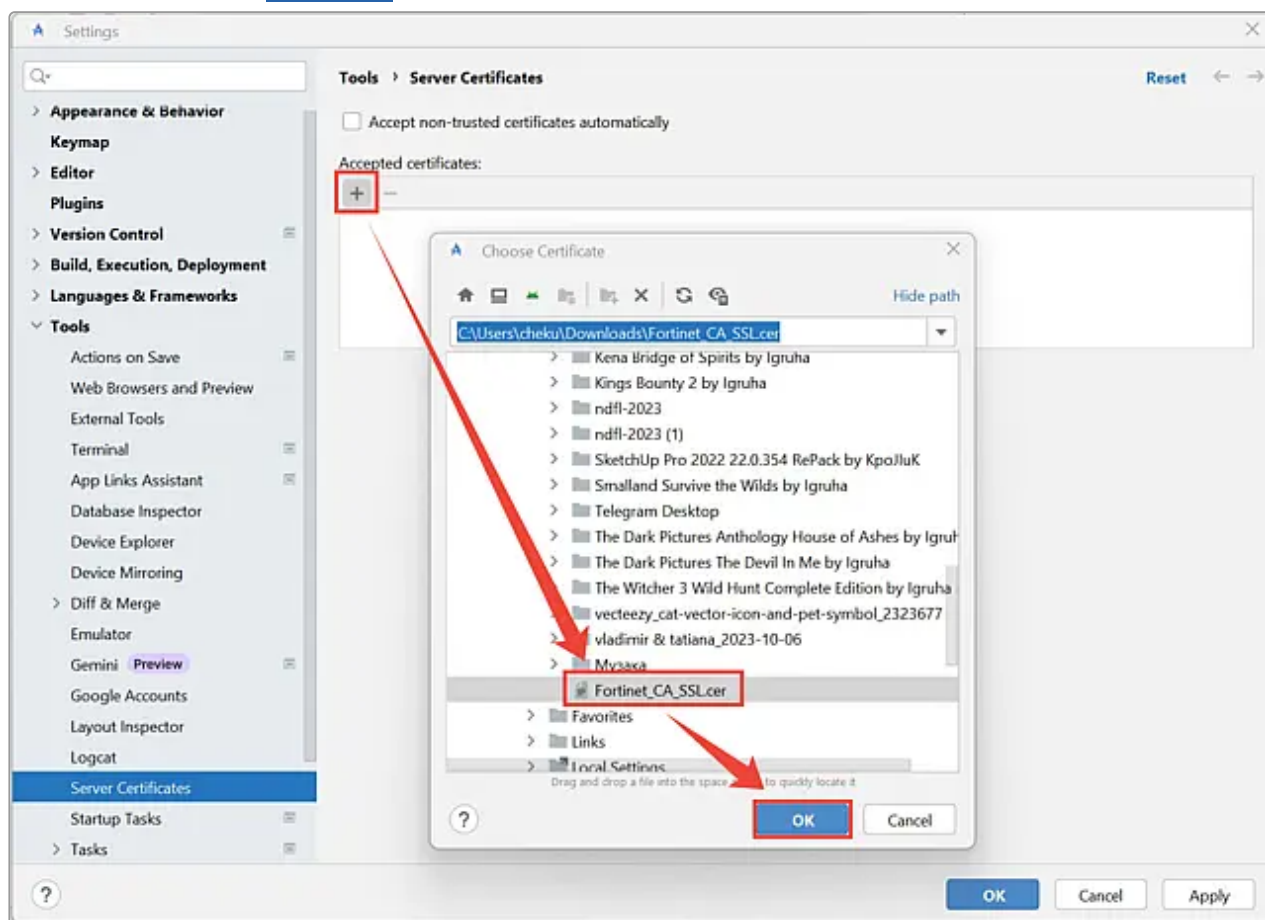
Шаг 1. Добавление сертификата в Android Studio

 Данный шаг позволит избежать ошибок "Server's certificate is not trusted" при попытке сборки приложения.

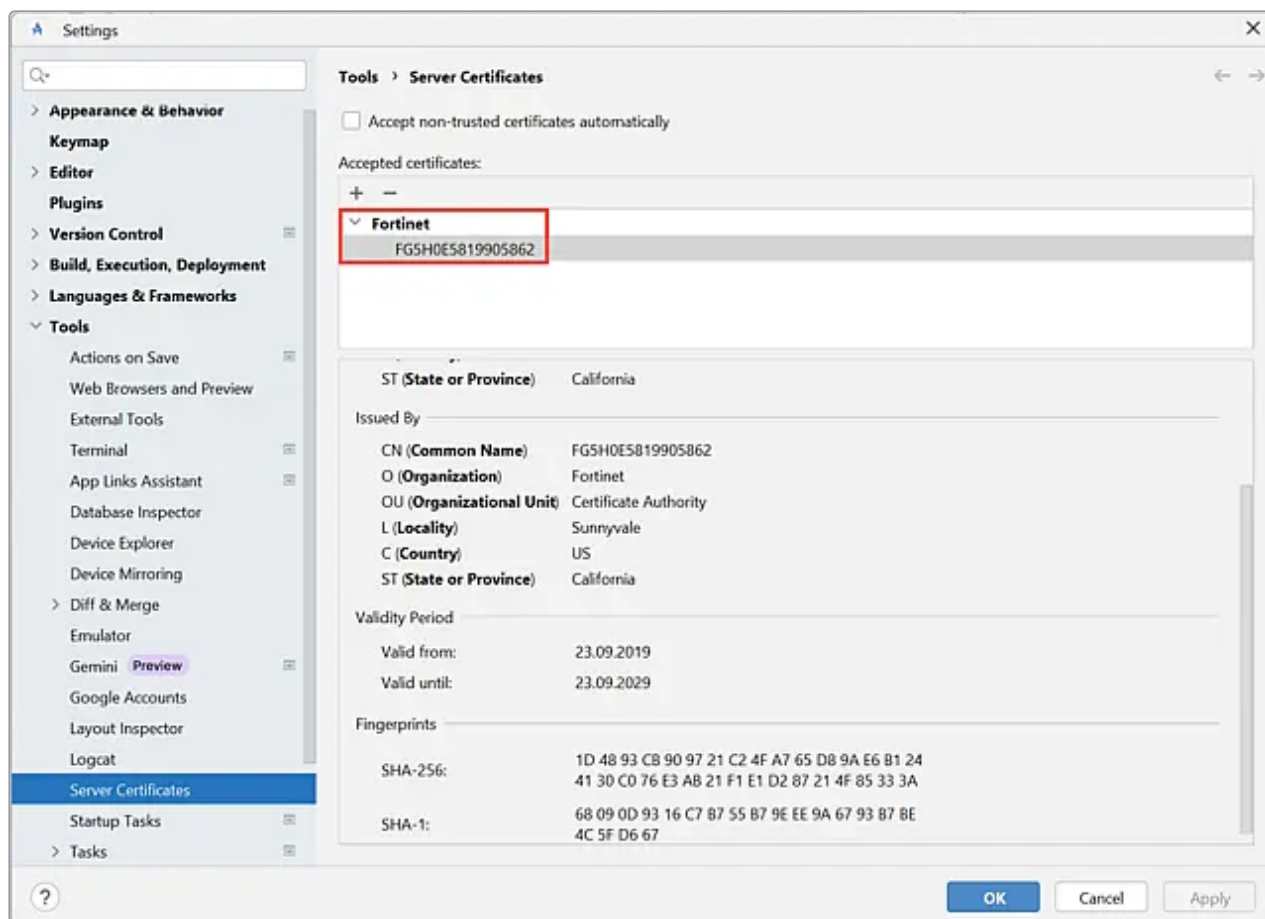
1. Перейдите в раздел **File** → **Settings** → **Tools** → **Server Certificates**.



2. По кнопке "+" добавьте [сертификат](#) в общий список.



3. Проверьте, что сертификат добавлен в список, нажмите кнопку "Apply", а затем кнопку "OK".



Шаг 2. Импорт сертификата в хранилище ключей

 Данный шаг позволит избежать проблем со скачиванием зависимостей для сборки приложения.

1. Скопируйте или скачайте [сертификат](#) из [шага 1](#).
2. Скопируйте файл в директорию <каталог, где хранится Android Studio>/jbr/lib/security.
3. Откройте терминал или командную строку (терминал Git Bash не подходит) в директории, в которую положили сертификат, и выполните команду:

```
"<каталог, где хранится Android Studio>/jbr/bin/keytool" -keystore "  
<каталог, где хранится Android Studio>/jbr/lib/security/cacerts" -  
importcert -alias Fortinet_CA_SSL -file Fortinet_CA_SSL.cer
```

4. После успешного выполнения команды появится запрос пароля на импорт сертификата. Введите стандартный пароль хранилища - changeit.
5. Далее будет задан вопрос про доверие сертификату. Введите yes.
6. Перезапустите Android Studio. Зависимости будут загружены после перезапуска.

Настройка гибридного отладчика

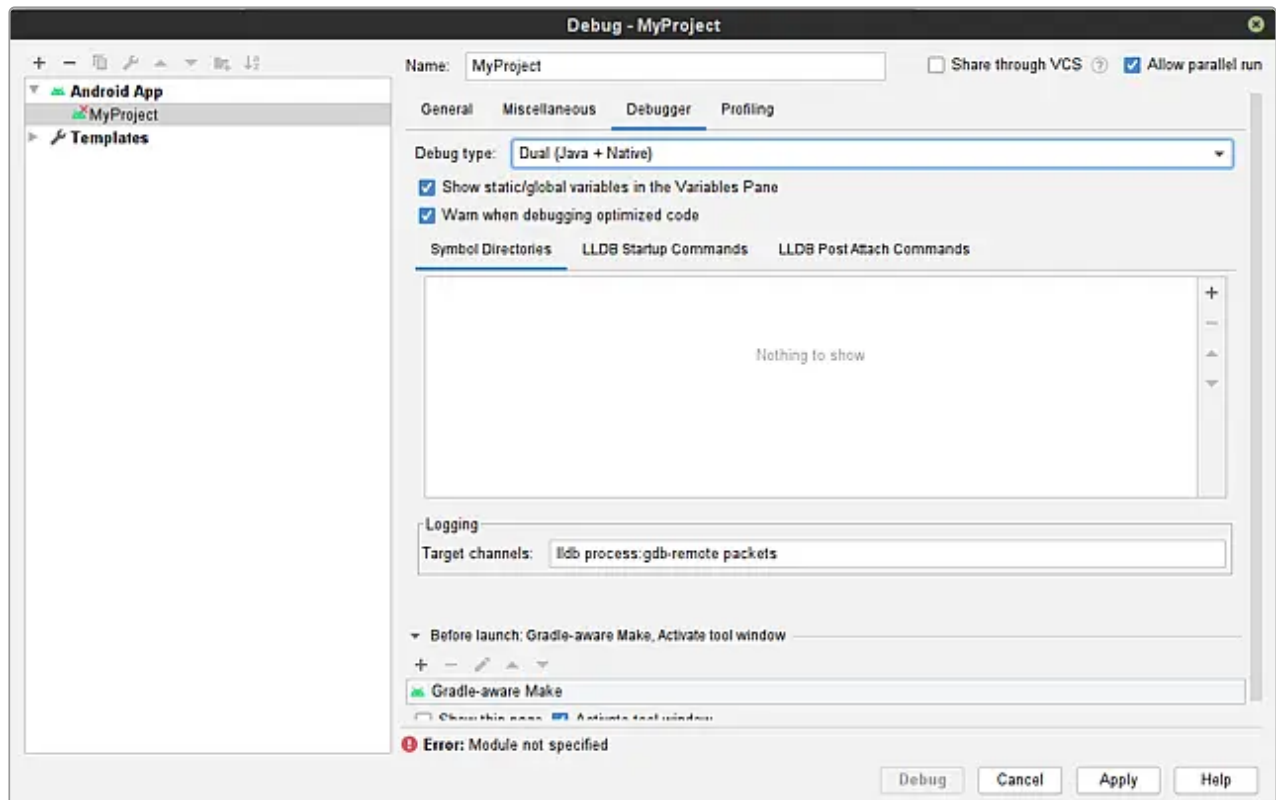
 Данная настройка требуется, только если это сборка Android вместе с контроллером.

В **Android Studio** используется гибридный отладчик, который обеспечивает возможность совместной отладки C++ и Java кода.

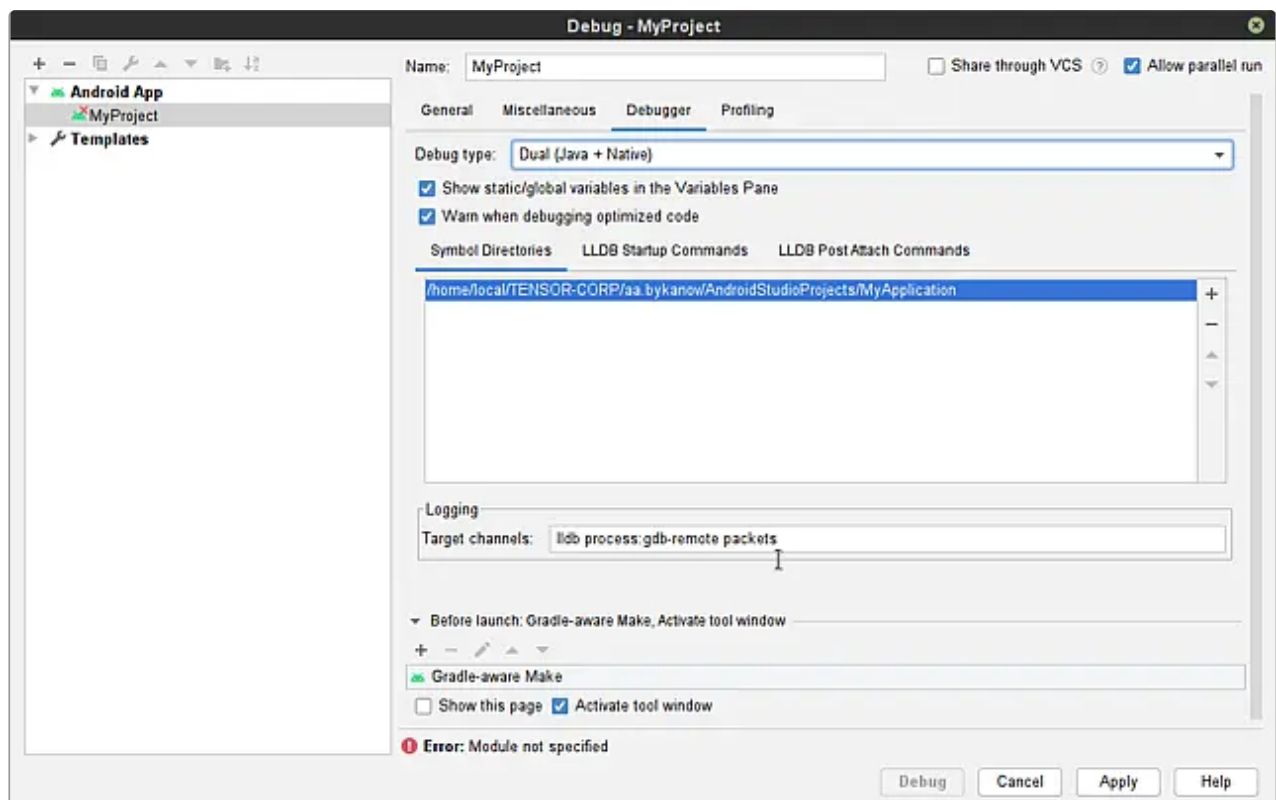
Java код отлаживается специальным Java-отладчиком, C/C++ код обрабатывается отладчиком LLDB. По умолчанию гибридная отладка не активирована и без дополнительной настройки отлаживать C++-код не получится.

1. В окне проекта перейдите в меню **Run** → **Edit configurations...**
2. В диалоговом окне **Run/Debug Configurations** нажмите "+" и выберите **Android App** для добавления новой конфигурации отладчика.

3. Задайте имя конфигурации, например **MyProject** и перейдите на вкладку **Debugger**. Для атрибута **Debug Type** выберите **Dual (Java + Native)**.



Далее на вкладке **Symbol Directories** укажите директорию, где **Android Studio** будет искать отладочные символы для библиотек. Укажите значение вида `$WORKSPACE_ROOT/app/build/intermediates/merged_native_libs/`, где `$WORKSPACE_ROOT` — путь до корня git-репозитория для View-приложения СБИС.



4. Дополнительно, если вы работаете в RedHat, выполните следующее:

- Установите пакет [sbis-development](#).
- Установите пакет **patchelf** с помощью команды в терминале:

```
sudo yum install patchelf
```
- В директории **AndroidStudio** найдите каталог **LLDBFrontend**, откройте терминал и выполните команду:

```
patchelf --replace-needed libstdc++.so.6 /opt/gcc-  
9.1.1/lib64/libstdc++.so.6 LLDBFrontend
```

После этого **Android Studio** сможет отлаживать C++ код приложения.